CoRet: Improved Retriever for Code Editing

Fabio J. Fehr, Prabhu Teja Sivaprasad, Luca Franceschi, Giovanni Zappella

Boost Code Retrieval for Code Editing with

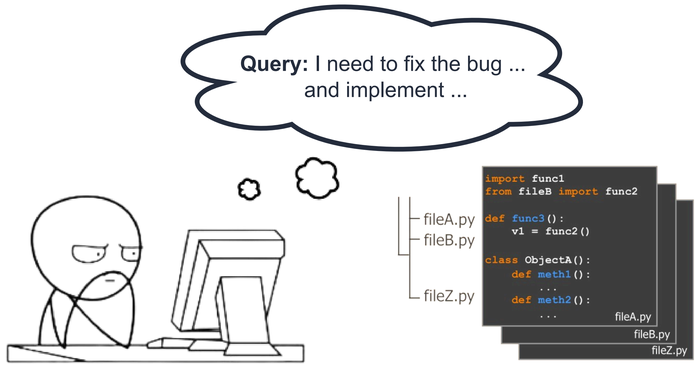
Repo Hierarchy-Aware Chunking

Call Graph Context augmentation

Likelihood loss based training

Check out the paper!

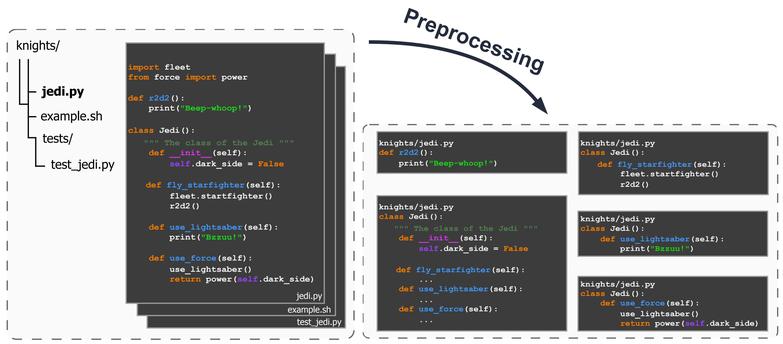
Code Editing Retrieval Problem



You — Code Repository

Which parts of the repo should you retrieve for editing?

Code Chunks with Repo-Hierarchy

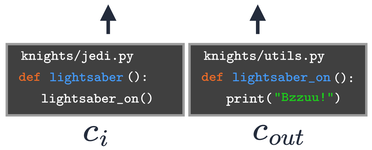


Code Repository — Code Chunks c_i

The code repo is split into semantically succinct unit we called **code chunks**.
We include **repo-hierarchy** structure by including the file path string.

Embedding with Call Graph Context

$$c_i \; ; \; \texttt{[DOWN]} \; ; \; c_{out}$$



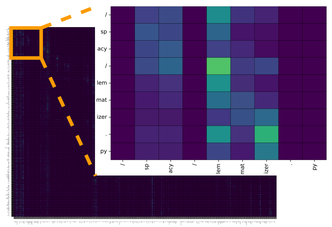
The chunk of interest c_i is concatenated as context with the out-going chunk c_{out} from the call graph and special token `[DOWN]`.

Call graph improves multi-chunk retrieval

	SWE Verified			LCA		
Model	@5	@20	MRR	@5	@20	MRR
CodeSage S	0.34	0.51	0.35	0.26	0.34	0.28
CoRet − CG	0.52	0.69	0.52	**0.32**	0.41	0.45
CoRet − CG + file	**0.54**	0.69	0.52	0.29	0.38	0.44
CoRet	**0.54**	**0.71**	**0.53**	**0.32**	**0.47**	**0.47**

CoRet: Fine-tuned CodeSage S (130M parameters).
SWE Verified: Software Engineering Benchmark (Verified subset).
LCA: Long Code Arena (Bug localisation task).

Repo-hierarchy is important

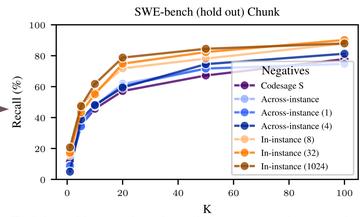


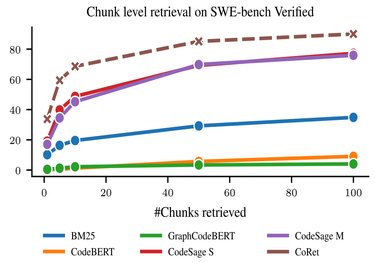
Training with Likelihood Loss

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i}^{N} \frac{1}{\mathcal{C}_i^*} \sum_{c^* \in \mathcal{C}_i^*} \log \frac{\exp(\boldsymbol{q}_i \cdot \boldsymbol{c}^*)}{\exp(\boldsymbol{q}_i \cdot \boldsymbol{c}^*) + \sum_{c \in \mathcal{B}} \exp(\boldsymbol{q}_i \cdot \boldsymbol{c})}$$

N = Number of repo instances i, $\mathcal{C}^*$ = Set of ground truth code chunks c^*, $\boldsymbol{q}$ = Natural Language query,
$\mathcal{B}$ = Random negative sample in the same repo instance.

Negatives from the same repo are best



Training with negatives from the same repo instance improve over negatives across repo instances (standard *in-batch* negatives).

Recall +15 percentage points